

Security Assessment Report



Aave V3.8

October 2026

Prepared for Aave Labs

Table of Contents

Project Summary **3**
Project Scope 3
Project Overview 3
Protocol Overview 3
Assessment Methodology **5**
Threat and Security Overview **6**
Findings Summary 9
Severity Matrix 9
Detailed Findings **10**
Informational Severity Issues 11
I-01: Transient storage dependency makes the upgrade unsafe on chains without EIP-1153 11
Disclaimer **12**
About Certora **12**

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave V3.8	https://github.com/aave-dao/aave-v3-origin	c7d5775ae5f7046d4285d5355ef092d5b24f377a	Solidity

Project Overview

This document describes the security review of **Aave V3.8**. The work was undertaken from **October 13th, 2026** to **October 20th, 2026**.

The following contract list is included in our scope:

src/contracts/instances/PoolInstance.sol
src/contracts/protocol/pool/Pool.sol
lib/solidity-utils/lib/openzeppelin-contracts-upgradeable/lib/openzeppelin-contracts/contracts/utils/ReentrancyGuardTransient.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Aave V3.8 is a defence-in-depth release of the Aave V3 lending protocol. It changes no feature, no interface and no storage slot. It hardens one thing earlier releases relied on implicitly: that no Pool action can be entered while another is in flight.

The reentrancy guard lives in transient storage, so it adds no proxy state. Actions that move funds or touch reserve and position accounting hold the lock while they run. Flash loans and token rescue instead only check that the lock is free, because their callbacks legitimately

re-enter guarded functions. The aToken's settlement callback into the Pool is guarded for the first time, so a token transfer can no longer settle while a locked action is in flight.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Threat and Security Overview

Scope and Trust Assumptions

The review examined the V3.8 reentrancy guard for logic errors and exploitable gaps, concentrating on the points where the Pool hands control to other contracts and expects to get it back: token transfers inside supply, withdraw, borrow, repay and liquidation, the flash loan receiver callback, and the aToken's `finalizeTransfer` callback. Concretely it covered entry point coverage, lock semantics in transient storage behind a proxy, reachability of guarded functions from legitimate callbacks, read-only reentrancy, the token rescue path, storage layout and initialisation on upgrade, and per-chain opcode support. Analysis combined manual review, the repository's Foundry tests and live-chain probes. The unchanged lending logic, including oracle integration, liquidation incentives and the economic parameters that govern them, was not reassessed and remains covered by prior reviews of Aave V3.

Because security analysis is inherently creative and any line may introduce a vulnerability, the following sections outline the primary categories of risk evaluated rather than an exhaustive list.

The following are trust assumptions. They were not verified in depth, but they must hold for safe operation.

- **Blockchain liveness and integrity.** Blocks are produced, valid transactions are accepted, fees stay reasonable, and MEV does not critically harm users.
- **Admin and governance integrity.** Proxy admin and ACL role holders are secure and not malicious.

Listed asset behaviour is deliberately not assumed. The guard exists so that a token which gains a callback, through an upgrade or otherwise, cannot use it against the Pool.

Security Considerations and Threat Analysis

Entry point coverage. The guard is only as good as the set of functions it covers. The threats evaluated follow.

- A fund-moving or accounting function reachable without a check, including the `deposit` alias, the permit variants, the `OnBehalfOf` position-manager functions and the configurator-only setters.
- `multicall` bypassing the lock. It delegatecalls into the same proxy, so each sub-call takes and releases the lock in turn.
- The L2 calldata wrappers double-locking by calling an already guarded public function internally.

Lock semantics. The flag lives in transient storage and the Pool runs behind a proxy. The threats evaluated follow.

- The lock not being visible to a re-entrant call that arrives from another contract, such as the aToken calling back into the Pool.
- The flag surviving past the call that set it, or not being cleared when that call reverts.
- The guard contract carrying storage of its own and shifting every Pool slot behind it in the inheritance order.
- Deployment to a chain that does not execute the transient opcodes, which would revert every guarded function.

Legitimate re-entry. Several flows must keep working. The threats evaluated follow.

- Flash loan receivers losing the ability to supply, repay, liquidate, move aTokens or open a nested flash loan, because `flashLoan` holds a lock during the callback.
- `rescueTokens` losing the ability to recover aTokens sent to the Pool, because the aToken transfer re-enters `finalizeTransfer`.
- Periphery contracts that chain an aToken `transferFrom` with a Pool action, such as the WETH gateway and the Stata token, reverting on the second step.
- A flash loan being started from inside a guarded action through a token hook, which `onlyNotReentrant` refuses.

Accounting integrity under reentrancy. The threats evaluated follow.

- An aToken transfer settling through `finalizeTransfer` while a liquidation or withdrawal has the sender's collateral state half-updated.

- Supply or borrow caps bypassed by re-entering during the validation window.
- Virtual balance, indexes or treasury accrual applied out of order across nested actions.
- Read-only reentrancy, where a view returns inconsistent state during an external call. The review checked that state writes precede the token transfer in supply, withdraw and liquidation, and that the flash loan reduces the virtual balance before the callback.

Upgrade and integration surface. The threats evaluated follow.

- Initialisation on the revision bump running incorrectly or not at all.
- Bytecode growth against the contract size limit, in particular for the L2 Pool.
- Gas overhead on every guarded action.

Adversarial Incentives

Attacks are costly. The incentive categories considered follow.

- **Griefing.** Make a legitimate integration revert by driving it into the guard, for example a position manager that calls back into the Pool mid-action.
- **Accounting corruption.** Re-enter during a supply, withdrawal or liquidation to leave reserve or user state inconsistent, then profit from the inconsistency on the next action.
- **Collateral extraction.** Use a callback to pass a health check against collateral that a later step of the same action is about to remove, ending the transaction undercollateralised.
- **Liveness denial.** Block flash loan composability or treasury recovery by locking where only a check was needed, or deploy to a chain where the guard cannot execute.
- **Direct theft.** Extract funds belonging to the protocol or other users. No path was identified. Every external call made by a guarded action runs with the lock held, and the two functions that run without it make no accounting changes of their own.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	1	1	-
Total	1	1	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
I-01	Transient storage dependency makes the upgrade unsafe on chains without EIP-1153	Informational	Acknowledged

Informational Severity Issues

I-01: Transient storage dependency makes the upgrade unsafe on chains without EIP-1153

Files:

Pool.sol

Description:

The new reentrancy guard uses transient storage, so every guarded Pool function now executes `TLOAD` and `TSTORE`. On chains that have not implemented `EIP-1153`, these opcodes are invalid. The implementation can still be deployed and initialized on such a chain, since neither the constructor nor `initialize` uses transient storage, but every guarded Pool function reverts afterwards. Metis, where a legacy Aave market exists, and Polygon zkEVM are chains that lack EIP-1153.

Recommendations:

Add the `nonReentrant` modifier to `PoolInstance.initialize`, so that upgrading on a chain without EIP-1153 reverts at upgrade time instead of disabling the market afterwards.

Customer Response:

Acknowledged. User-facing and configuration entry points are guarded, so even if initialization succeeds, any incompatibility with the target chain's EVM would cause subsequent operations to revert, making the issue immediately apparent and allowing corrective action to be taken.

Furthermore, contract upgrades undergo a multi-step review process involving the DAO's technical and security service providers, including validation that the new implementation is compatible with the target chain's EVM.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.